

Evidencia N°2

Plan de seguridad del Proyecto Integrador

1. Antes de comenzar:

- Descargar este documento de la Evidencia N°2, como Documento Word, así completan las consignas directamente y no tienen que empezar un documento desde cero.
- Un solo compañero debe subir el mismo documento descargado y subirlo a un Drive e invitar a los compañeros del mismo equipo.
- Leer y estudiar el material correspondiente de la Unidad II, que se encuentra en la sección de Material Obligatorio, en el Aula Virtual.

2. Requisitos de entrega:

- Nombrar el archivo *Documentación_Ciberseguridad_App[NombreDelProyecto]*, reemplazando *[NombreDelProyecto]* por el nombre real de su proyecto integrador (por ejemplo, *Documentación_Ciberseguridad_SangreYa*).
- Todos los integrantes deben participar y editar dicho documento.
- Una vez terminado y revisado por todo el equipo, descargarlo en formato PDF y agregar el enlace en la Wiki del repositorio del Proyecto Integrador.
- Un solo compañero del equipo deberá compartir el enlace del documento colaborativo en la entrega en el Aula Virtual (configurar el Acceso general como Comentarista para la posterior revisión de la Docente).

3. Datos de los integrantes y del proyecto

- Integrantes:
- Gonzalo Javier Velasco DNI 34988724 - gonzalo.velasco@mi.unc.edu.ar
- Guillén, Jonathan Eduardo DNI 34440292 - jonathan.guillen@mi.unc.edu.ar
- Corvalán Alejandro DNI 35576754 - alejandromcorva@gmail.com

- Salvo Daniela Paola DNI 31041526 - daniela7salvo@gmail.com
- Gonzalez Martinez Roni Duncan DNI 44195660 - roniduncang@gmail.com
- Gerardo Lauro Romero DNI 35782538 gerardolauroromero@gmail.com
- Enlace al repositorio de Programación Web (frontend y backend del primer cuatrimestre): <https://github.com/ISPC-TSDWAD/ModPWeb-Dev6>
- Enlace al repositorio de Proyecto Integrador:
<https://github.com/ISPC-TSDWAD/Mobile-Testing-Cyberseguridad-Dev-6>
- Descripción y objetivos del proyecto:

Edutools es una plataforma web y móvil diseñada para docentes, maquettadores y asesores pedagógicos. Su objetivo principal es optimizar y estandarizar la creación de recursos didácticos digitales mediante un sandbox de diseño interactivo permitiendo componer, personalizar y exportar componentes visuales estructurados (organizadores temáticos, cuadros de llamada de atención y bloques destacados) compatibles con aulas virtuales, asegurando accesibilidad, consistencia pedagógica y persistencia segura de datos mediante un backend en Django REST Framework y un frontend en Angular.

4. Datos y principios CIA

Enumeren qué datos de sus usuarios maneja el proyecto, y de esos, cuáles consideran sensibles (por ejemplo, datos financieros, datos personales, datos de salud). Para cada dato sensible que identifiquen, comenten si debería estar protegido y cómo lo protegerían.

Datos y Principios Cia (Confidencialidad, Integridad y Disponibilidad)

- 1). Enumeración de datos de usuario que maneja el proyecto:

1. **Credenciales de acceso:** Nombre de usuario (username) y contraseña (password).
2. **Tokens de sesión:** Token de acceso (access) y Token de refresco (refresh) en formato JWT.
3. **Datos de identidad personal (PII):** Nombre de pila (first_name), apellido (last_name) y correo electrónico (email).
4. **Datos de privilegios y control:** Rol del sistema (rol:"admin","docente","asesor","maquetador"), pertenencia al staff (is_staff) y estado de la cuenta (is_active).
5. **Datos de autoría y recursos didácticos:** Contenidos HTML de los recursos (contenido), enlaces web (url), identificador del creador (creado_por), condición de plantilla base (es_plantilla_base) e historial de creación/actualización

2). Datos sensibles identificados, justificación y mecanismo de protección:

A. Contraseña de los usuarios

- **¿Por qué es sensible?:** Constituye la llave primaria de acceso al sistema. Su filtración vulnera la **Confidencialidad** y permite la suplantación de identidad.
- **¿ Debería estar protegida?:** Si, de forma crítica y obligatoria.
- **¿cómo la protegemos?**
 - **En reposo (Base de datos):** Nunca guardarla en texto plano. En Django ya se utiliza el algoritmo de hashing criptográfico seguro **PBKDF2 con SHA-256** y aleatoria(make_password()),lo que mitiga ataques de diccionario y de tablas arcoiris.
 - **En tránsito (Red):** Obligar el uso estricto de **HTTPS** en producción para evitar la interceptación de tráfico (Man-in-the-Middle)entre el cliente y el servidor.

- **En la entrada:** Reforzar los validadores de complejidad (mínimo 8 caracteres, combinación de mayúsculas, minúsculas, números y símbolos).

B. Correo electrónico y Datos Personales (email, first_name, last_name)

- **¿Por qué es sensible?:** Son datos de identificación personal regulados por normativas de privacidad. Su exposición no autorizada puede dar lugar a campañas de phishing, spam o ingeniería social dirigida (spear phishing).
- **¿Debería estar protegido?: Si.**
- **¿Cómo lo protegemos?:**
 - Restringir el principio de **Confidencialidad**: el correo y los nombres completos solo deben ser visibles para el propio usuario dueño del perfil y para los administradores del sistema.
 - Eliminar la exposición de estos datos en endpoints públicos o listados generales mediante el uso de serializadores restringidos en Django REST Framework.

C. Rol y Nivel de privilegios (rol, is_ staff)

- **¿Por qué es sensible?:** Si un atacante logra modificar su rol o adjudicarse is staff: true, se produce una escalada de privilegios que compromete la **Integridad** total del sistema, permitiéndole borrar o alterar recursos y usuarios de toda la institución.
- **¿Debería estar protegido?: Si**
- **¿Cómo lo protegemos?:**
 - Marcando estos campos como de **sólo lectura** (read_only=True) en los serializadores accesibles por el usuario común.
 - Validando en el backend que únicamente un usuario con permisos de administrador autenticado pueda modificar la tabla de roles y privilegios, desacoplandolo de las peticiones de actualización de perfil de usuario (PATCH/api/users/me).

D. Tokens de Sesión (JWT access y refresh)

- **¿Por qué es sensible?:** Quien posea el token de acceso representa la identidad del usuario ante el backend sin necesidad de conocer su contraseña.
- **¿Debería estar protegido?:** Sí.
- **¿Cómo lo protegemos?:**
 - Firmando los tokens con una clave secreta robusta (SECRET_KEY) que resida fuera del código fuente.
 - Configurando tiempos de expiración reducidos para el token de acceso (reduciéndolo de 24 horas a 15-30 minutos).
 - En la web, evitar guardarlos en localStorage (vulnerable a ataques XSS) y utilizar cookies seguras con atributos HttpOnly, Secure y SameSite=Strict . En aplicaciones móviles (Android), almacenarlos en memoria protegida mediante EncryptedSharedPreferences.

E. Disponibilidad de los datos y del servicio

- **¿Por qué es sensible?:** Los dos principios anteriores protegen que los datos no se filtren ni se alteren, pero un recurso didáctico al que un docente no puede acceder cuando lo necesita está igualmente comprometido. La Disponibilidad es el tercer principio de la tríada CIA y es el que atacan tanto el ransomware, que cifra la base de datos y la vuelve inaccesible, como la denegación de servicio (DoS/DDoS), que satura el servidor hasta que deja de responder a todos los usuarios legítimos.
- **¿Debería estar protegida?:** Sí.
- **¿Cómo la protegemos?:**
 - Respallos periódicos de la base de datos MySQL, almacenados en una ubicación separada del propio servidor de la aplicación, de modo que un cifrado por ransomware sobre ese servidor no alcance también a las copias. Prever además una prueba de

restauración, porque un backup que nunca se restauró no es un backup verificado.

- Versionado de los recursos didácticos, para que una modificación o un borrado accidental no implique la pérdida definitiva del trabajo del docente.
- Limitación de tasa de solicitudes (throttling) en los endpoints de la API, que además de frenar los ataques de fuerza bruta sobre el login acota el impacto de una saturación por volumen de peticiones.
- Despliegue detrás de un proveedor con mitigación de DDoS y monitoreo de disponibilidad del servicio, para detectar una caída sin depender de que la reporte un usuario.

5. Vulnerabilidades del Backend

Reúnanse como equipo y revisen juntos, con el backend a mano, cada ítem de la siguiente lista.

Para cada punto, además de describir lo que encontraron, indiquen a cuál o cuáles categorías de OWASP Top 10 corresponde (puede ser más de una, según lo que vieron en el material): Pérdida de Control de Acceso, Configuración de Seguridad Incorrecta, Fallas en la Cadena de Suministro de Software, Fallas Criptográficas, Inyección, Diseño Inseguro, Fallas de Autenticación, Fallas en la Integridad del Software o de los Datos, Fallas en el Registro y Monitoreo, o Manejo Inadecuado de Condiciones Excepcionales.

a. Secretos escritos directamente en el código

Revisen si hay contraseñas, claves de acceso o datos de conexión a la base de datos escritos directamente en algún archivo de código, en vez de en un archivo

de variables de entorno que no se suba al repositorio. Por ejemplo: 'PASSWORD': '1234' escrito directo en el archivo de configuración, en vez de 'PASSWORD': `os.getenv('DB_PASSWORD')`. Para revisarlo, busquen en los archivos de configuración de su proyecto cualquier contraseña o clave escrita literalmente entre comillas.

- **Resultado de lo encontrado:**

Al revisar el archivo Backend/core/[settings.py](#), se observa que se implementó la librería python-dotenv para cargar la variable SECRET_KEY y las credenciales de base de datos (DB_NAME, DB_USER, DB_PASSWORD) desde un archivo .env.

Sin embargo, en el historial reciente del repositorio existió un archivo .env que fue subido al control de versiones de Git con credenciales reales y que luego fue eliminado dejando el riesgo residual en el historial de commits. Además, si la variable de entorno SECRET_KEY no se define en el servidor, Django inicializa con valor None o falla sin advertencia previa.

- **Categoría de OWASP Top 10 correspondiente:**
- **A02 - Configuración de Seguridad Incorrecta (Security Misconfiguration)**
- **A04 - Fallas Criptográficas (Cryptographic Failures)**

b. Modo de desarrollo activo

Revisen si el proyecto tiene activado, en el ambiente donde está corriendo ahora, algún modo de desarrollo o depuración que muestre información técnica detallada ante un error. Por ejemplo: `DEBUG = True` escrito fijo en el archivo de configuración, en vez de depender de una variable que se pueda cambiar a False al desplegar. Para revisarlo, provoquen un error a propósito (por ejemplo, pidiendo algo que no existe con la siguiente URL: `http://127.0.0.1:8000/noExiste`) y vean si la respuesta muestra código interno, rutas del servidor o datos de configuración, en vez de un mensaje simple.

- **Resultado de lo encontrado:**

En Backend/core/[settings.py](#) la directiva DEBUG depende de la variable de entorno:

```
DEBUG = os.getenv('DEBUG', 'False').strip().lower in ('1', 'true', 'yes').
```

Sin embargo, en el archivo de plantilla provisto en el proyecto (Backend/.env_modelo) figura explícitamente DEBUG=True.

Si al desplegar la aplicación en un servidor de prueba o de producción se toma como base esta plantilla y se olvida cambiar este valor, cualquier visitante que consulte una ruta inexistente (por ejemplo: <http://127.0.0.1:8000/noExiste>) o provoque un error 500 recibirá una página técnica interactiva de Django que expone fragmentos de código fuente, rutas absolutas del disco, versiones exactas de las librerías instaladas y configuraciones internas del sistema.

- **Categoría de OWASP Top 10 correspondiente:**
- **A02 - Configuración de Seguridad Incorrecta /(Security Misconfiguration)**
- **A10 - Manejo Inadecuado de condiciones Excepcionales (Mishandling of Exceptional Conditions)**

c. Orígenes permitidos

Revisen si el backend permite pedidos desde cualquier sitio web, o solo desde el dominio de su propia aplicación. Por ejemplo: una configuración como "permitir todos los orígenes", en vez de una lista específica de direcciones autorizadas. Para revisarlo, busquen en la configuración del servidor algo relacionado con "CORS" u "orígenes permitidos".

- **Resultado de lo encontrado:**

En BACKEND/Core/[Settings.py](#), el backend cuenta con la siguiente lógica:

- Python
- `_cors_origins = [o.strip() for o in os.getenv('CORS_ALLOWED_ORIGINS', "").split(',') if o.strip()]`

- if _cors_origins:
- CORS_ALLOWED_ORIGINS = _cors_origins
- CORS_ALLOW_ALL_ORIGINS = False
- else:
- CORS_ALLOW_ALL_ORIGINS = True # <--- Vulnerabilidad
- En Backend/.env_modelo, CORS_ALLOWED_ORIGINS se encuentra vacía por defecto. Por ende, el sistema activa automáticamente CORS_ALLOW_ALL_ORIGINS = True. Esto permite que cualquier página web externa en internet pueda realizar peticiones HTTP cruzadas contra la API del backend, pudiendo inducir al navegador de un usuario autenticado a realizar acciones indebidas o filtrar datos sensibles.

- **Categoría de OWASP Top 10 correspondiente:**
- **A02 - Configuración de Seguridad Incorrecta (Security Misconfiguration)**

d. Endpoints sin autenticación

Revisen si alguno de los endpoints que devuelven datos de usuarios, pedidos o carritos responde igual sin haber iniciado sesión. Por ejemplo: pedir la lista completa de usuarios o pedidos desde el navegador o Postman o Swagger, sin haber iniciado sesión, y recibirla igual. Para revisarlo, prueben acceder a esos endpoints sin loguearse y vean si el sistema les niega el acceso o les devuelve los datos igual.

- **Resultado de lo encontrado:**

En backend /core / [settings.py](#) la configuración del diccionario REST_FRAMEWORK solamente define DEFAULT_AUTHENTICATION_CLASSES (JWT), pero **no define** DEFAULT_PERMISSION_CLASSES .

En Django Rest Framework , la política global por defecto en ausencia de esta configuración es AllowAny (permisiva). Si bien en las vistas actuales (UsuarioViewSet y RecursoViewSet) se agregaron permisos a nivel de clase (IsAdminOrOwner e

IsAuthenticated), cualquier nuevo endpoint o vista que un integrante del equipo agregue en el futuro sin declarar explícitamente `permission_classes` quedará abierto al público por omisión.

Asimismo, los endpoints `/api/token` y `/api/token/refresh` no tiene implementada limitación de tasa de solicitudes (rate limiting / throttling), lo que expone el login a ataques automatizados de pulverización de contraseñas (password spraying) y fuerza bruta.

- **Categoría de OWASP Top 10 correspondiente:**
- **A01 - Pérdida de control de acceso (Broken access control)**
- **A07 - Fallas de autenticación (identification and authentication failures)**

e. Control de acceso por ID

Estando logueados, revisen si cambiar el número o ID de un recurso en la URL o en el pedido (por ejemplo, el de un carrito o un pedido que no es el suyo) les permite verlo o modificarlo igual. Por ejemplo: cambiar `/api/pedidos/7` por `/api/pedidos/8` y obtener los datos de un pedido ajeno. Para revisarlo, prueben cambiar ese número a mano y observen la respuesta.

- **Resultado de lo encontrado:**

Al auditar backend/ users/ [permissions.py](#) en la clase `IsAdminOrOwner`, se observa :

```
python def has_object_permission (self, request, view, obj ) :
```

```
    if request.user and request.user.is_staff:
```

```
        return True
```

```
    If request.method in permissions.SAFE_METHODS:
```

```
        return True
```

```
    return obj == request.user
```

Esto genera una vulnerabilidad directa de Referencia directa insegura a objetos (IDOR/ **BOLA**) donde un usuario autenticado con un rol común de docente puede solicitar `get/api/users/1` o `get/api/users/2` cambiando el numero de ID en el url y obtener la ficha de datos personales completa de otros usuarios o administradores, violando el aislamiento entre cuentas.

Adicionalmente en `RecursoViewSet`, la lógica de protección de plantillas base solo comprueba el rol "Asesor", permitiendo que otros roles no administrativos puedan aplicar métodos patch o put sobre recursos ajenos.

- **Categoría de OWASP Top 10 correspondiente:**
- **A01 - Perdida de control de acceso (Broken access control)**
- **A06 - Diseño inseguro (insecure design)**

f. Dependencias reconocidas

Revisen si reconocen para qué sirve cada librería o paquete externo que usa su proyecto. Por ejemplo: encontrar en el archivo de dependencias dos o tres librerías que nadie del equipo recuerda haber agregado ni para qué sirven.

Para revisarlo, repasen el archivo donde su proyecto lista sus dependencias (`requirements.txt`, `package.json`, `pom.xml`, u otro según su lenguaje), y marquen las que no sepan identificar o no estén usando realmente.

- **Resultado de lo encontrado:**

Al auditar `backend/requirements.txt`, se identifican dependencias reconocidas para el funcionamiento del backend (Django, `django-rest-framework`, `mysqlclient`, `python-dotenv`, `django-cors-headers`, `asgiref`, `sqlparse`, `tzdata` y `PyJWT`).

Sin embargo, se detectó una anomalía en la gestión de paquetes: La librería `django-rest-framework-simplejwt` == 5.3.1 es duplicada (aparece en la línea 5 y en la línea 10). Además, no existe un archivo de bloqueo con firmas criptográficas de integridad (Hash de paquetes con `pip-compile` o `Pipfile.lock`), ni se ejecutan ninguna herramienta

de análisis estático continuo de vulnerabilidades en librerías de terceros como pip-audit, lo que expone el entorno a paquetes con vulnerabilidades conocidas (CVEs).

- **Categoría de OWASP Top 10 correspondiente:**
- **A03 - Fallas en la cadena de suministro de software (Software supply chain failures)**

6. Plan de Seguridad

A partir de lo que encontraron en el punto anterior, armen un plan concreto para cada ítem donde detectaron un problema. Este plan será la base de lo que van a implementar en el código durante la Evidencia N°3, así que piensen en soluciones que realmente puedan llevar a cabo con lo que van aprendiendo en la materia. Para cada ítem de la lista anterior donde hayan encontrado un problema, respondan:

a. Secretos escritos directamente en el código

- o ¿Qué van a hacer para solucionarlo?
 - Verificar que el archivo .env se mantenga permanentemente declarado en .gitignore, y agregar también los patrones *.env y .env.* para cubrir las variantes por entorno.
 - Purgar el archivo .env del historial de commits de Git mediante la herramienta git filter-repo o BFG Repo-Cleaner, y forzar la reescritura del repositorio remoto, dado que el borrado simple del archivo no elimina el riesgo residual.
 - Rotar, es decir regenerar, todas las credenciales que estuvieron expuestas: la clave secreta de Django y la contraseña del usuario de MySQL, asumiendo que deben considerarse comprometidas.
 - Implementar en el archivo de configuración una validación estricta de arranque, de manera que si la variable SECRET_KEY no existe o está vacía el servidor interrumpa el inicio lanzando la excepción

ImproperlyConfigured, en lugar de arrancar con un valor nulo o por defecto.

- Mantener el archivo .env_modelo únicamente con claves de ejemplo y sin valores reales, documentando qué debe completar cada integrante en su entorno local.

b. Modo de desarrollo activo

- o ¿Qué van a hacer para solucionarlo?
 - Mantener y reforzar el valor por defecto seguro de la directiva DEBUG en el archivo de configuración, de modo que la ausencia de la variable de entorno nunca active el modo depuración y el sistema quede siempre en falso por omisión.
 - Corregir el archivo .env_modelo, cambiando DEBUG=True por DEBUG=False, y agregar un comentario que aclare que solo debe activarse en el entorno local de desarrollo.
 - Condicionar la lista ALLOWED_HOSTS para que en producción admita únicamente el dominio del proyecto, y nunca el comodín que acepta cualquier host.
 - Crear manejadores de error personalizados para los códigos 404 y 500 que devuelvan una respuesta JSON genérica, evitando por completo la fuga de trazas de depuración, rutas absolutas del disco y versiones de las librerías instaladas.
 - Configurar el módulo LOGGING de Django para que los errores queden registrados del lado del servidor, de modo que dejar de mostrarlos al usuario no implique perder visibilidad sobre las fallas.

c. Orígenes permitidos

- ¿Qué van a hacer para solucionarlo?

- o Eliminar de forma definitiva la rama condicional que activa automáticamente CORS_ALLOW_ALL_ORIGINS, invirtiendo el criterio: si no hay orígenes configurados, la lista debe quedar vacía y denegar todo, en lugar de abrir la API a cualquier sitio de Internet.
- o Definir una lista blanca explícita en el archivo .env de cada entorno. En desarrollo local, los orígenes del frontend Angular en localhost y 127.0.0.1 sobre el puerto 4200; en producción, únicamente el dominio HTTPS final donde se aloje el frontend.
- o Configurar CSRF_TRUSTED_ORIGINS con esos mismos dominios y habilitar el envío de credenciales solo para ellos.
- o Activar en producción las directivas SECURE_SSL_REDIRECT, SESSION_COOKIE_SECURE y CSRF_COOKIE_SECURE, para forzar que todo el tráfico y las cookies viajen sobre HTTPS.

d. Endpoints sin autenticación

- o ¿Qué van a hacer para solucionarlo?
 - Establecer una política restrictiva por defecto en la configuración de Django REST Framework, declarando IsAuthenticated en DEFAULT_PERMISSION_CLASSES . De este modo todo endpoint nuevo nace protegido y exige token JWT, salvo que se indique lo contrario de forma explícita.
 - Declarar permission_classes con AllowAny solamente en las vistas que deben ser públicas, es decir login, refresh y registro, dejando documentado el motivo en cada caso.
 - Incorporar las clases de limitación de tasa AnonRateThrottle y UserRateThrottle, fijando un límite acotado para las peticiones anónimas, del orden de cinco por minuto, y un límite más amplio para las autenticadas.
 - Aplicar una limitación de tasa específica sobre los endpoints de obtención y refresco de token mediante ScopedRateThrottle, para

frenar los ataques de fuerza bruta y de pulverización de contraseñas.

- Registrar en el log los intentos de autenticación fallidos, consignando usuario y marca temporal pero nunca la contraseña, a fin de poder detectar patrones de ataque.
- Verificar el resultado probando cada endpoint sin token desde Postman o Swagger: la respuesta esperada debe ser 401 Unauthorized.

e. Control de acceso por ID

- o ¿Qué van a hacer para solucionarlo?
 - Corregir la clase IsAdminOrOwner en el archivo de permisos, eliminando la excepción que habilita los métodos seguros a cualquier usuario autenticado. El permiso debe conceder acceso únicamente si el solicitante es administrador o si el objeto consultado es su propio registro.
 - Canalizar la consulta del perfil propio exclusivamente a través del endpoint /api/users/me/, que deriva el usuario del token JWT y no de un identificador manipulable en la URL.
 - Restringir el queryset de UsuarioViewSet para que un usuario no administrador solo obtenga su propio registro, evitando que el listado completo de usuarios quede expuesto.
 - En RecursoViewSet, validar en las operaciones de actualización y borrado que el recurso pertenezca al usuario solicitante, o que este sea administrador, y reservar la modificación de las plantillas base a los roles asesor y admin.
 - Comprobar la corrección con una prueba manual: autenticarse con un rol común de docente e intentar acceder al perfil de otro usuario

cambiando el identificador en la URL, y modificar un recurso ajeno.

La respuesta esperada es 403 Forbidden o 404 Not Found.

▪

f. Dependencias reconocidas

- o ¿Qué van a hacer para solucionarlo?
 - Limpiar el archivo requirements.txt eliminando la línea duplicada de django-rest-framework-simplejwt, y revisar si la librería pyjwt se utiliza de forma directa o llega solo como dependencia transitiva.
 - Fijar todas las versiones y generar un archivo de bloqueo con hashes de integridad mediante la herramienta pip-compile, de modo que la instalación sea reproducible y verificable.
 - Incorporar la herramienta pip-audit al flujo de trabajo del equipo y ejecutarla antes de cada entrega, para detectar librerías con vulnerabilidades reportadas en las bases de datos de CVE.
 - Documentar en el README del backend el propósito concreto de cada dependencia, para que ningún integrante mantenga paquetes que no sepa justificar.
 - Aplicar el mismo criterio al frontend Angular, ejecutando la auditoría de npm sobre el archivo package.json.

Nota: si en algún ítem del punto anterior no encontraron ningún problema, pueden mencionar que no han encontrado ningún problema.

7. Autenticación

Este punto depende de en qué etapa esté su proyecto, respondan la opción que corresponda a su equipo:

- **Si ya implementaron JWT u otro método de autenticación en su backend:**
describan cómo lo armaron. ¿Qué información guarda el token? ¿Por cuánto

tiempo es válido antes de vencer? ¿Qué reglas de complejidad exige el sistema para las contraseñas al registrarse (cantidad mínima de caracteres, combinación de letras, números y símbolos)?

- **Si todavía no lo implementaron:** este es el momento de empezar a diseñarlo, aunque sea a nivel de idea general. Un token es una credencial que el servidor le entrega al usuario después de un login exitoso, y que la aplicación vuelve a presentar en cada pedido posterior para demostrar que ya inició sesión, sin tener que enviar la contraseña de nuevo cada vez. ¿Qué método de autenticación piensan usar (JWT es el más común, pero no el único)? ¿Por qué les parece la opción adecuada para su proyecto? ¿Qué reglas de complejidad piensan exigir para las contraseñas (cantidad mínima de caracteres, combinación de letras, números y símbolos)?

Cómo lo armamos

El sistema utiliza la especificación JSON Web Token a través de la librería oficial `django-rest-framework-simplejwt`. La obtención de credenciales está expuesta en la ruta `POST /api/token/`, atendida por la vista personalizada `CustomTokenObtainPairView`, vinculada a su vez al serializador `CustomTokenObtainPairSerializer`.

Cuando el usuario ingresa credenciales válidas desde Angular o desde la futura aplicación móvil Android, el servidor valida la contraseña contra el hash almacenado y emite dos tokens firmados digitalmente con el algoritmo HMAC-SHA256. El token de acceso se envía en la cabecera `Authorization`, con el esquema `Bearer`, en cada petición a un endpoint protegido. El token de refresco se utiliza en la ruta `POST /api/token/refresh/` para obtener un nuevo token de acceso cuando el anterior vence, sin necesidad de volver a pedir la contraseña.

La verificación de la firma se realiza en cada petición mediante la clase `JWTAuthentication`, declarada en la configuración de Django REST Framework. Al tratarse de un esquema sin estado, el servidor no almacena sesiones en memoria, lo que resulta adecuado para una API consumida en simultáneo por un cliente web y uno móvil.

Por qué elegimos JWT

La elección responde a que el proyecto debe atender dos clientes distintos sobre la misma API: el frontend web en Angular y la aplicación móvil en Android. Un esquema basado en sesiones y cookies del lado del servidor obligaría a mantener estado por cada cliente y resulta incómodo de consumir desde una aplicación nativa. JWT, en cambio, permite que el servidor valide cada petición verificando únicamente la firma del token, sin consultar un almacén de sesiones, lo que simplifica la arquitectura y facilita el escalado. Además, al incorporar claims personalizadas con el rol del usuario, el cliente puede adaptar la interfaz sin realizar una consulta adicional al backend.

Qué información guarda el token

El token de acceso contiene los metadatos estándar de la especificación: el tipo de token, la fecha de vencimiento expresada en timestamp UNIX, la fecha de emisión y un identificador criptográfico único. Incluye además el identificador numérico del usuario en la base de datos MySQL.

A esto se suman dos claims personalizadas que inyecta nuestro serializador, para que el cliente conozca los permisos sin consultar la base de datos en cada vista. La primera indica si el usuario posee permisos de administrador. La segunda expresa su rol dentro del sistema, que puede ser admin, docente, asesor o maquetador, y se utiliza para condicionar las opciones disponibles en la interfaz gráfica.

El token de refresco, en cambio, contiene únicamente los metadatos de sesión y el identificador del usuario.

Cabe aclarar que el contenido de un JWT está codificado en Base64, no cifrado, por lo que cualquiera que intercepte el token puede leerlo. La firma garantiza integridad, es decir que nadie pueda alterarlo, pero no confidencialidad. Por ese motivo el token no contiene la contraseña ni datos personales sensibles, y su transporte debe realizarse siempre sobre HTTPS.

Por cuánto tiempo es válido antes de vencer

Según la configuración actual del proyecto, el token de acceso tiene una vigencia de un día, es decir 24 horas, y el token de refresco una vigencia de siete días.

Como propuesta de mejora para la Evidencia N°3, consideramos que 24 horas es un tiempo elevado desde el punto de vista de la seguridad, porque si el token es interceptado el atacante dispone de un día completo de acceso. Por lo tanto se reducirá la vigencia del token de acceso a 30 minutos, manteniendo los siete días del token de refresco, y se activarán la rotación de tokens de refresco y su inclusión en una lista negra tras cada rotación, de modo que sea posible invalidarlos ante un cierre de sesión o un robo de credenciales.

Qué reglas de complejidad exige el sistema para las contraseñas

Actualmente el backend aplica los cuatro validadores nativos de Django. El primero impide que la contraseña sea igual o muy similar al nombre de usuario, nombre, apellido o correo electrónico. El segundo exige una longitud mínima de ocho caracteres. El tercero compara la contraseña contra una lista negra de las claves más comunes y predecibles a nivel mundial. El cuarto rechaza las contraseñas formadas exclusivamente por dígitos.

Las contraseñas se almacenan **hasheadas** con el algoritmo PBKDF2 sobre SHA-256 y sal aleatoria, por lo que nunca se guardan ni se transmiten en texto plano.

Durante la auditoría detectamos una inconsistencia: el endpoint de cambio de contraseña valida únicamente que la nueva clave tenga al menos seis caracteres,



omitiendo los validadores globales. Esto permite fijar por esa vía una clave más débil que la exigida al momento de registrarse.

Como plan de mejora para la Evidencia N°3 se aplicará la validación completa también en el cambio de contraseña, se elevará la longitud mínima a diez caracteres y se incorporará un validador personalizado por expresión regular que exija de forma combinada al menos una letra mayúscula, una letra minúscula, un dígito numérico y un carácter especial.